Chapter 2: Numbering Systems

Introduction

Data types found in the registers of digital computers

- Numbers used in arithmetic computations
- Letters of the alphabet used in data processing
- Other discrete symbols used for specific purpose

Number Systems

- Base or Radix *r* system: uses distinct symbols for *r digits*
- Most common number system: Decimal, Binary, Octal, Hexadecimal
- Positional-value(weight) System: r^2 r $^1r^0.r^{-1}$ r^{-2} r^{-3}
  - Multiply each digit by an integer power of *r* and then form the sum of all weighted digits

Decimal Number System (Base-10 system)

- Decimal numbers are made of decimal digits:

  (0,1,2,3,4,5,6,7,8,9)

- But how many items does a decimal number represent?

  $8653 = 8x10^3 + 6x10^2 + 5x10^1 + 3x10^0$

- What about fractions?

  $97654.35 = 9x10^4 + 7x10^3 + 6x10^2 + 5x10^1 + 4x10^0 + 3x10^{-1} + 5x10^{-2}$

  - ✓ In formal notation -> $(97654.35)_{10}$

- Why do we use 10 digits, anyway?

Base-10 system Cont…

- Numbers are represented by a series of coefficients:

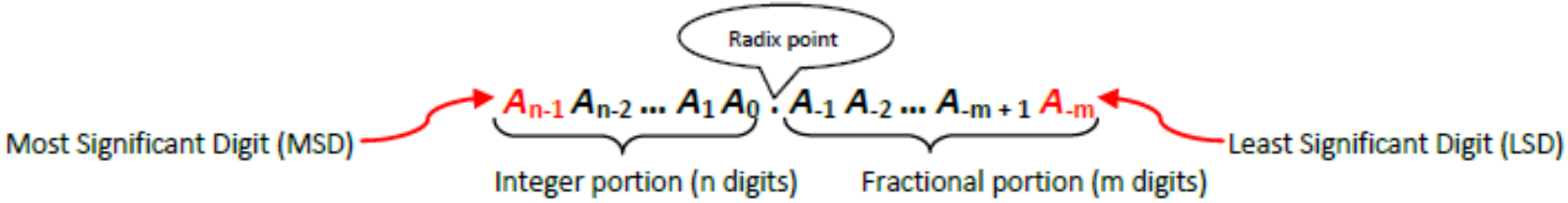
$$a^5\ a^4\ a^3\ a^2\ a^1\ a^0\ .\ a^{-1}\ a^{-2}\ a^{-3}$$

The coefficients a^j are any of the 10 digits (0, 1, 2, … , 9)

- In general, a number expressed in a base-r system has coefficients multiplied by powers of r:

$$a_n.r^n + a_{n-1}.r^{n-1} + \ldots + a_2.r^2 + a_1.r + a_0 + a_{-1}.r^{-1} + a_{-2}.r^{-2} + \ldots + a_{-m}.r^{-m}$$

The coefficients a_i range in value from 0 to r - 1.



Octal Number System (Base-8 system)

- Octal numbers are made of octal digits:

  (0,1,2,3,4,5,6,7)

- How many items does an octal number represent?

  $(4536)_8 = 4x8^3 + 5x8^2 + 3x8^1 + 6x8^0 = (2398)_{10}$

- What about fractions?

  $(465.27)_8 = 4x8^2 + 6x8^1 + 5x8^0 + 2x8^{-1} + 7x8^{-2}$

- Octal numbers don't use digits 8 or 9

Binary Number System (Base-2 system)

- Radix (r) = 2
- A binary numbers are expressed with a string of 1's and 0's.
- Example: $(11010.01)_2$ can be interpreted using power series as

$$(11010.01 = 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2} = (26.25)_{10}$$

- Digits in a binary number are called bits (**B**inary dig**IT**s).

Hexadecimal Number System (Base-16 system)

- The letters of the alphabet are used to supplement the 10 decimal digits when the base of the number is greater than 10. For example, in the hexadecimal (base-16)
- Radix (r) = 16
- Symbols = 0 through r-1 = 0 through 16-1 = {0, 1, 2…9, A, B, C, D, E, F}
- A hexadecimal numbers are expressed with a strings of symbols defined above, and with possibly, a *hexadecimal point*.
- The decimal equivalent of a hexadecimal number can be found by expanding the number into a power series with a base of 16.

Example

1: $(4D71B.C6)_{16}$

$$= 4 \times 16^4 + D \times 16^3 + 7 \times 16^2 + 1 \times 16^1 + B \times 16^0 + C \times 16^{-1} + 6 \times 16^{-2}$$
$$= 4 \times 16^4 + 13 \times 16^3 + 7 \times 16^2 + 1 \times 16^1 + 11 \times 16^0 + 12 \times 16^{-1} + 6 \times 16^{-2}$$
$$= (317211.7734375)_{10}$$

2: $(B65F)_{16}$

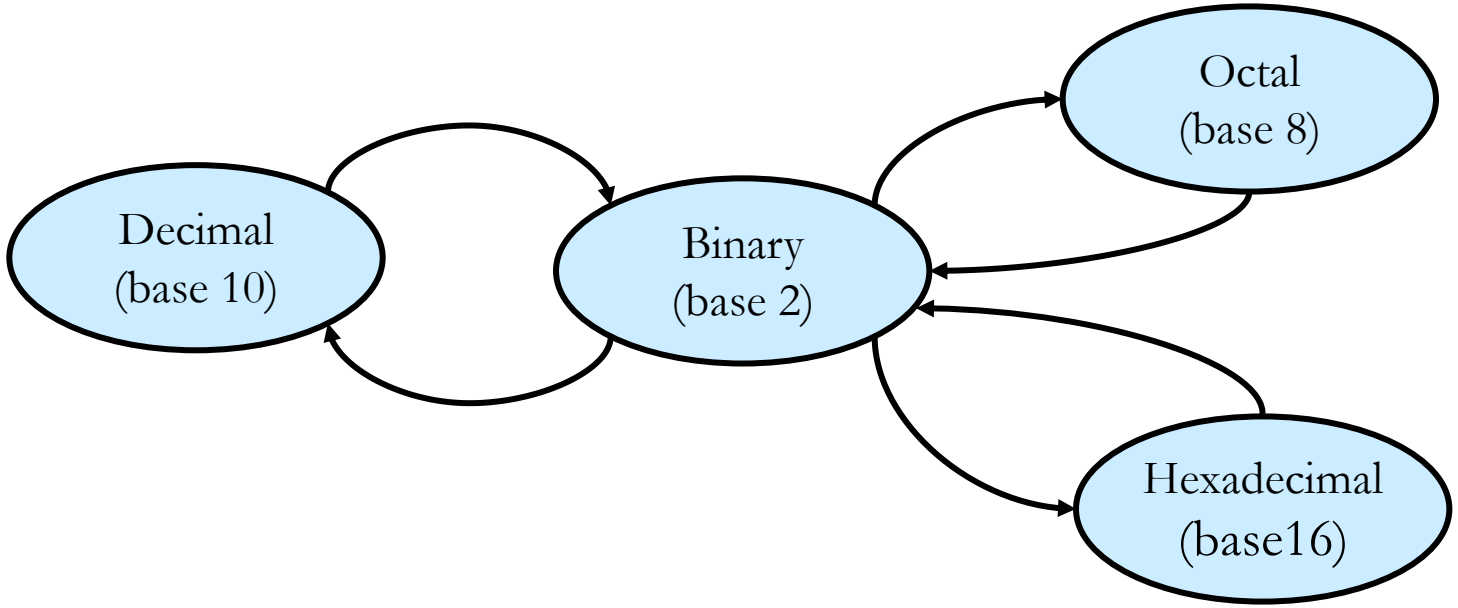
$$= 11 \times 16^3 + 6 \times 16^2 + 5 \times 16^1 + 15 \times 16^0$$
$$= (46687)_{10}$$

3: $(\mathrm{B65F})_{16}$ in binary

$$= 1011\ 0110\ 0101\ 1111$$

Number-Base Conversion

- Base-r system can be binary
  - ✓ (r=2), octal (r=8), hexadecimal (r=16), base-60 system or any other
- For decimal system as destination of conversion, we just use power series explained earlier with varying *r* and sum the result according to the arithmetic rules of base-10 system.



Example

Convert decimal 41 to binary

	Integer Quotient		Remainder	Coefficient
41/2	20	+	½	$a_0=1$
20/2	10	+	0	$a_1=0$
10/2	5	+	0	$a_2=0$
5/2	2	+	½	$a_3=1$
2/2	1	+	0	$a_4=0$
½	0	+	½	$a_5=1$

- Divide decimal number by the base (i.e. 2)
- The remainder is the lowest-order digit
- Repeat first two steps until no divisor remains.
- The *coefficients* of the desired binary number are obtained from the *remainders*.

Therefore, the answer is $(41)_{10} = (a_5\, a_4\, a_3\, a_2\, a_1\, a_0)_2 = (101001)_2$.

Decimal to Base 2 Conversion

Convert 179 into binary

179 / 2 = 89 remainder 1 (LSB)

/ 2 = 44 rem 1

/ 2 = 22 rem 0

/ 2 = 11 rem 0

/ 2 = 5 rem 1

/ 2 = 2 rem 1

/ 2 = 1 rem 0

/ 2 = 0 rem 1 (MSB)

Hence, $179_{10} = 10110011_2$

Convert Decimal 153 to Octal

Solution

- The required base *r* is 8.
- First, 153 is divided by 8 to give an integer quotient of 19 and a remainder of 1.
- Then 19 is divided by 8 to give an integer quotient of 2 and a remainder of 3.
- Finally, 2 is divided by 8 to give a quotient of 0 and a remainder of 2.
- This process can be conveniently manipulated as follows:

153	
19	1
2	3
0	$2 = (231)_8$

Fraction *from* Decimal *to* Another Base

Convert decimal $(0.625)_{10}$

Solution

For each digit position:

- ✓ Multiply decimal number by the base (e.g. 2)
- ✓ The integer is the highest-order digit
- ✓ Repeat first two steps until fraction becomes zero.

	Integer		**Remainder**	Coefficient
0.625 x 2 =	1	+	0.25	$a_{-1} = 1$
0.250 x 2 =	0	+	0.50	$a_{-2} = 0$
0.500 x 2 =	1	+	0	$a_{-3} = 1$

Answer $(0.625)_{10} = (0.a_{-1}\, a_{-2}\, a_{-3})_2 = (0.101)_2$

Other Examples

From binary to octal

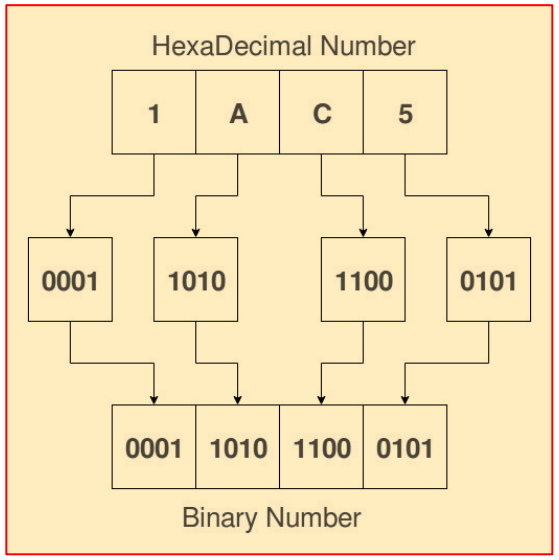
- ✓ Partitioning the binary number into groups of three digits each,
- ✓ Start from the binary point
- ✓ Assign corresponding octal digit

$$\begin{array}{lllllllllll} (10 & 110 & 001 & 101 & 011 & \cdot & 111 & 100 & 000 & 110)_2 = (26153.7406)_8 \\ 2 & 6 & 1 & 5 & 3 & & 7 & 4 & 0 & 6 \end{array}$$

From binary to hexadecimal

- ✓ Divide binary number into groups of *four* digits:

$$\begin{array}{lllllll} (10 & 1100 & 0110 & 1011 & \cdot & 1111 & 0010)_2 = (2C6B.F2)_{16} \\ 2 & C & 6 & B & & F & 2 \end{array}$$



Octal to binary

$$\begin{array}{lllllllll} (673.124)_8 = (110 & 111 & 011 & \cdot & 001 & 010 & 100)_2 \\ 6 & 7 & 3 & & 1 & 2 & 4 \end{array}$$

Hexadecimal digit to binary

$$\begin{array}{lllll} (306.D)_{16} = (0011 & 0000 & 0110 & \cdot & 1101)_2 \\ 3 & 0 & 6 & & D \end{array}$$

Example

Convert 190 to base 3

- ✓ First, we take the number that we wish to convert and divide it by the radix in which we want to express our result.
- ✓ Record the quotient and the remainder.
- ✓ Continue in this way until the quotient is zero.
- ✓ Our result, reading from bottom to top is:

$$190_{10} = 21001_3$$

3	190	1
3	63	0
3	21	0
3	7	1
3	2	2
	0	

Numbers with Different Bases

Decimal (base 10)	Binary (base 2)	Octal (base 8)	Hexadecimal (base 16)
00	0000	00	0
01	0001	01	1
02	0010	02	2
03	0011	03	3
04	0100	04	4
05	0101	05	5
06	0110	06	6
07	0111	07	7
08	1000	10	8
09	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

Addition and Subtraction of Nondecimal Numbers

Addition

X	190	1 0 1 1 1 1 1 0
Y	141	1 0 0 0 1 1 0 1
___+		_____
X+Y	331	1 0 1 0 0 1 0 1 1

Binary subtraction: (borrow, difference bits)

Minuend	X	229	1 1 1 0 0 1 0 1
Subtrahend	Y	46	0 0 1 0 1 1 1 0
_____-			_____
Difference	X-Y	183	1 0 1 1 0 1 1 1

- ✓ Use binary subtraction to compare numbers. If X-Y produces a borrow out at the most significant bit, then X is less than Y.

Signed Integer Representation

Left most bit (MSB) is sign bit

- 0 means positive
- 1 means negative

There are three ways in which signed binary integers may be expressed:

- Signed magnitude
- One's complement
- Two's complement

Signed Magnitude

$01010101_2 = +85_{10}$

$01111111_2 = +127_{10}$

$00000000_2 = +0_{10}$

$11010101_2 = -85_{10}$

$11111111_2 = -127_{10}$

$10000000_2 = -0_{10}$

- In an 8-bit word, *signed magnitude* representation places the absolute value of the number in the 7 bits to the right of the sign bit.

  +18 = 00010010

  -18 = 10010010

- Problems
  - ✓ Need to consider both sign and magnitude in arithmetic
  - ✓ Two representations of zero (+0 and -0)

- Signed magnitude representation is easy for people to understand, but it requires complicated computer hardware.

Example 9

Using signed magnitude binary arithmetic, find the sum of 75 and 46.

Solution

```
        1 1 1
0   1001011
0 + 0101110
0   1111001
```

Example

Using signed magnitude binary arithmetic, find the sum of 107 and 46.

Solution

```
1
        1   1 1 1
0     1 1 0 1 0 1 1
0   + 0 1 0 1 1 1 0
0     0 0 1 1 0 0 1
```

We see that the carry from the seventh bit overflows and is discarded, giving us the erroneous result: 107 + 46 = 25

Example

Using signed magnitude binary arithmetic, find the sum of - 46 and – 25

Solution

```
        1 1
1    0101110
1  + 0011001
1    1000111
```

Because the signs are the same, all we do is add the numbers and supply the negative sign when we are done.

Example

Using signed magnitude binary arithmetic, find the sum of 46 and -25.

Solution

$$
\begin{array}{cccccccc}
 & & & 0 & 2 & & 0 & 2 \\
0 & & 0 & \cancel{1} & 0 & 1 & \cancel{1} & 0 \\
1 & + & 0 & 0 & 1 & 1 & 0 & 0 & 1 \\
\hline
0 & & 0 & 0 & 1 & 0 & 1 & 0 & 1
\end{array}
$$

The sign of the result gets the sign of the number that is larger.

One's Complement

- Here, negative values are represented by some difference between a number and its base.
- It is the diminished radix complement of a non-zero number N in base r with d digits is $(r^d - 1) - N$
- In the binary system, this gives us one's complement. It amounts to little more than flipping the bits of a binary number.

- For example, using 8-bit one's complement representation:

  + 3 is: 00000011

  - 3 is: 11111100

- Negative values are indicated by a 1 in the high order bit.
- Complement systems are useful because they eliminate the need for subtraction.
  - ✓ The difference of two values is found by adding the minuend to the complement of the subtrahend

Example

Using one's complement binary arithmetic, find the sum of 48 and - 19

Solution

```
1   1 1
    00110000
    11101100
    --------
    00011100
           + 1
    --------
    00011101
```

The carry bit is "carried around" and added to the sum.

19 in binary is 00010011.
-19 in one's complement is: 11101100.

Two's Complement

- One's complement system still has the problem of two different representations for zero: positive zero and negative zero.
- Two's complement solves this problem.
- Two's complement is the radix complement of the binary numbering system; the *radix complement* of a non-zero number N in base r with d digits is $r^d - N$.

✓ To express a value in two's complement representation:
  - If the number is positive, just convert it to binary and you're done.
  - If the number is negative, find the one's complement of the number and then add 1.

+3 =	00000011
+2 =	00000010
+1 =	00000001
+0 =	00000000
-1 =	11111111
-2 =	11111110
-3 =	11111101

Example

Using two's complement binary arithmetic, find the sum of 48 and - 19.

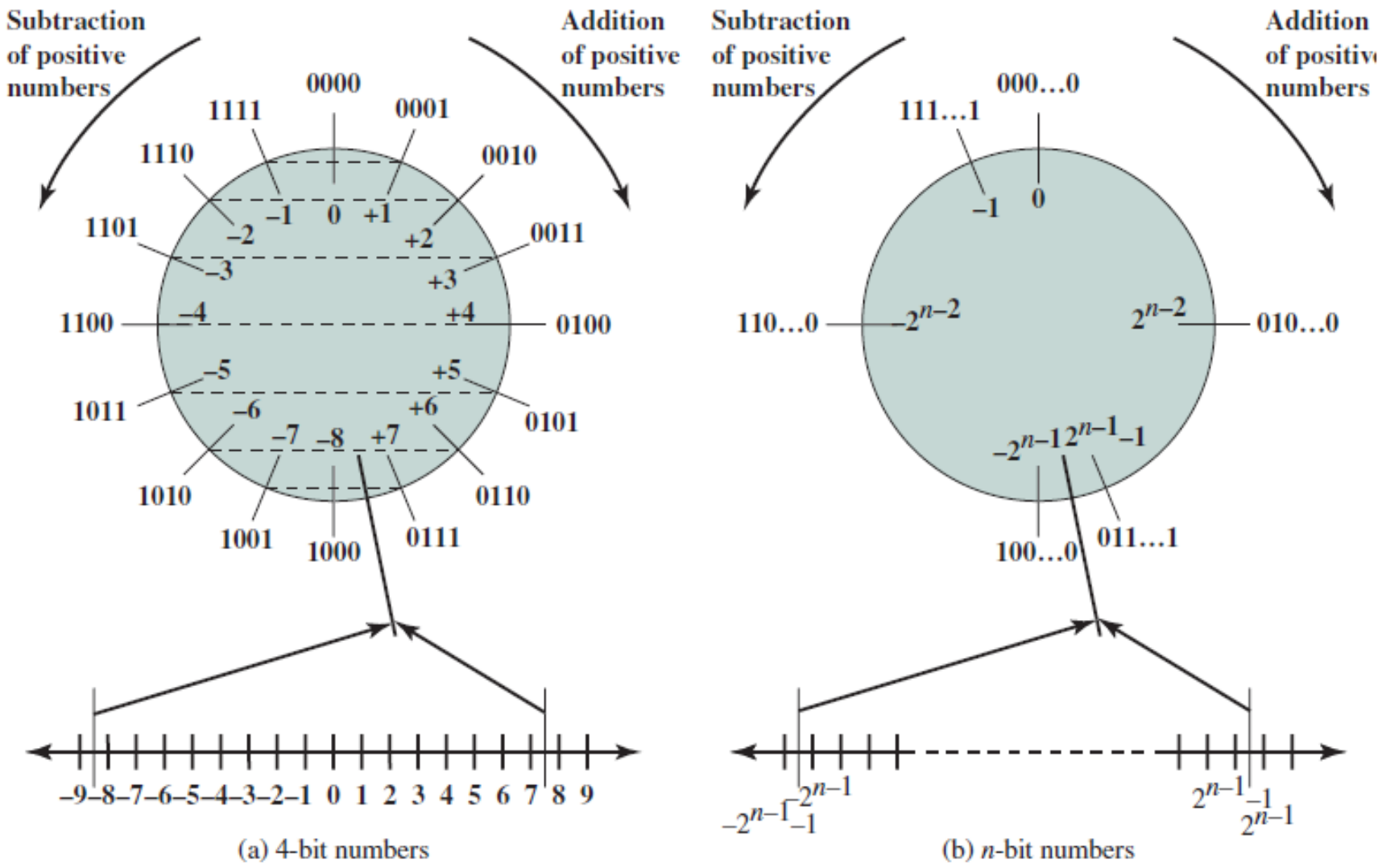
Solution

```
1
    1 1
    00110000
  + 11101101
  ----------
    00011101
```

With two's complement arithmetic, all we do is add our two binary numbers. Just discard any carries emitting from the high order bit.

19 in binary is: 00010011
-19 using one's complement is: 11101100
-19 using two's complement is: 11101101

Geometric Depiction of Twos Complement Integers



Overflow

- In complement arithmetic, an overflow condition is easy to detect.
- Overflow into the sign bit does not always mean that we have an error

Example:

- Using two's complement binary arithmetic, find the sum of 107 and 46.

```
  1 1   1 1 1
  0 1 1 0 1 0 1 1
+ 0 0 1 0 1 1 1 0
-----------------
  1 0 0 1 1 0 0 1
```

- We see that the nonzero carry from the seventh bit overflows into the sign bit, giving us the erroneous result: 107 + 46 = -103.

Rule for detecting signed two's complement overflow

- When the "carry in" and the "carry out" of the sign bit differ, overflow has occurred.
- If the carry into the sign bit equals the carry out of the sign bit, no overflow has occurred.

Example:

- Using two's complement binary arithmetic, find the sum of 23 and -9.

- We see that there is carry into the sign bit and carry out. The final result is correct: 23 + (-9) = 14.

```
1 ← 1 1 1   1 1 1
    00010111
  + 11110111
  ---------
    00001110
```

Binary Codes

Binary-Coded Decimal Code (BCD)

- Any discrete element of information distinct among a group of quantities can be represented by a binary code.
- Binary-coded decimal (BCD) contains at least four bits arranged into 10 distinct combinations.
- A number with *k* decimal digits will require 4*k* bits in BCD. E.g.

$$396 = 0011\ 1001\ 0110$$

  - ✓ Each group of 4 bits represents one decimal digit.

$$(185)_{10} \Rightarrow (0001\ 1000\ 0101)_{BCD} \Rightarrow (10111001)_2$$

In this example, the BCD value has 12 bits to encode the characters of the decimal value, but the equivalent binary number needs only 8 bits.

Decimal Symbol	BCD Digit
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

BCD Addition

- Since each digit does not exceed 9, the sum cannot be greater than 9 + 9 + 1 = 19, with the 1 being a previous carry.
- Binary sum will produce a result in the range from 0 to 19. In binary, this range will be from 0000 to 10011, but in BCD, it is from 0000 to 1 1001, with the first (i.e., leftmost) 1 being a carry and the next four bits being the BCD sum.

4	0100	4	0100	8	1000
+5	0101	+8	+1000	+9	1001
9	1001	12	1100	17	10001
			+0110		+0110
			10010		10111

- ✓ When the binary sum is equal to or less than 1001 (without a carry), the corresponding BCD digit is correct.
- ✓ The addition of $6 = (0110)_2$ to the binary sum converts it to the correct digit and also produces a carry as required.
- ✓ This is because a carry in the most significant bit position of the binary sum and a decimal carry differ by 16 - 10 = 6.

Example

Consider the addition of 184 + 576 = 760 in BCD:

Solution

1	1		
0001	1000	0100	184
+0101	0111	0110	+576
0111	10000	1010	
	0110	0110	
0111	**0110**	**0000**	**760**

Gray Code

- Also known as Reflected Binary Code (RBC)
- 2 successive values differ in only 1 bit
- Used to reduce the switching operation of machines

- Steps
  - ✓ Record the MSB as it is
  - ✓ Add the MSB to the next bit, record the sum, and neglect the carry – or use the XOR
  - ✓ Repeat the process

Cont…

1 bit in the code group changes in going from one number to the next.

E.g. from 7 to 8,

0100 to 1100.

Only the first bit changes, from 0 to 1; the other three bits remain the same.

For Binary numbers change from 7 to 8

0111 to 1000,

which causes all four bits to change values.

Gray Code	Decimal Equivalent
0000	0
0001	1
0011	2
0010	3
0110	4
0111	5
0101	6
0100	7
1100	8
1101	9
1111	10
1110	11
1010	12
1011	13
1001	14
1000	15

ASCII Character Code

- American Standard Code for Information Interchange (ASCII)

- An alphanumeric character set is a set of elements that includes the 10 decimal digits, the 26 letters of the alphabet, and a number of special characters.

- A set contains
  - ✓ between 36 and 64 elements - capital letters (six bits)
  - ✓ between 64 and 128 elements - both uppercase and lowercase letters included (seven bits).

- Uses seven bits to code 128 characters
  - ✓ designated by b_1 through b_7, with b_7 the most significant bit.

Cont…

	$b_7b_6b_5$							
$b_4b_3b_2b_1$	000	001	010	011	100	101	110	111
0000	NUL	DLE	SP	0	@	P	`	p
0001	SOH	DC1	!	1	A	Q	a	q
0010	STX	DC2	“	2	B	R	b	r
0011	ETX	DC3	#	3	C	S	c	s
0100	EOT	DC4	$	4	D	T	d	t
0101	ENQ	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	‘	7	G	W	g	w
1000	BS	CAN	(	8	H	X	h	x
1001	HT	EM	)	9	I	Y	i	y
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	;	K	[	k	{
1100	FF	FS	,	<	L	\	l	|
1101	CR	GS	–	=	M	]	m	}
1110	SO	RS	.	>	N	^	n	~
1111	SI	US	/	?	O	–	o	DEL

- 94 graphic characters
  - ✓ 26 uppercase letters (A through Z),
  - ✓ 26 lowercase letters (a through z),
  - ✓ 10 numerals (0 through 9),
  - ✓ 32 special printable characters, such as %, *, and $.

- Example

  The letter *S*, = 1010011 (column 101, row 0011).

Cont…

- The 34 control characters are designated in the ASCII table with abbreviated names.

NUL	Null	DLE	Data-link escape
SOH	Start of heading	DC1	Device control 1
STX	Start of text	DC2	Device control 2
ETX	End of text	DC3	Device control 3
EOT	End of transmission	DC4	Device control 4
ENQ	Enquiry	NAK	Negative acknowledge
ACK	Acknowledge	SYN	Synchronous idle
BEL	Bell	ETB	End-of-transmission block
BS	Backspace	CAN	Cancel
HT	Horizontal tab	EM	End of medium
LF	Line feed	SUB	Substitute
VT	Vertical tab	ESC	Escape
FF	Form feed	FS	File separator
CR	Carriage return	GS	Group separator
SO	Shift out	RS	Record separator
SI	Shift in	US	Unit separator
SP	Space	DEL	Delete

Error-Detecting Code

- To detect errors, an eighth bit is sometimes added to the ASCII character to indicate its parity.

- A *parity bit* is an extra bit included with a message to make the total number of 1's either even or odd.

	With even parity	**With odd parity**
ASCII A = 1000001	01000001	11000001
ASCII T = 1010100	11010100	01010100